1. Executive Summary

- **Overview:** This project involved deploying a highly available web application on AWS using a multi-AZ architecture.
- **Findings:** The infrastructure successfully handled simulated traffic spikes, automatically scaling compute resources to maintain performance.
- **Challenges & Solutions:** Ensured seamless shared storage across dynamic instances by implementing Amazon EFS, and secured traffic flow using strict Security Group rules.

2. Introduction

- **Purpose:** To evaluate the performance, scalability, and resilience of the deployed AWS architecture under simulated load conditions.
- **Application Description:** A Python Flask web application hosted on Amazon Linux 2023, utilizing shared persistent storage and distributed traffic routing.

3. Deployment Strategy

- **Infrastructure:** Leveraged custom VPC for isolation, EC2 for compute, Auto Scaling Groups (ASG) for elasticity, Application Load Balancer (ALB) for traffic distribution, EFS for shared storage, and Route 53 for DNS routing.
- **Deployment Process:** Automated instance provisioning and application configuration using EC2 Launch Templates and User Data scripts.

4. Performance Metrics

- **Baseline Performance:** Initially, the architecture operated with a minimum of 2 EC2 instances, maintaining a baseline CPU utilization of under 10%.
- **Load Testing:** Conducted stress testing using the Linux stress utility (stress --cpu 4 timeout 300) to artificially spike CPU utilization to 100% and simulate high traffic.

5. Optimization Efforts

- **Initial Optimizations:** Configured Target Tracking Scaling Policies in the ASG to maintain an average CPU utilization of 50%.
- **Iterative Improvements:** Monitored CloudWatch metrics to ensure instances spun up within minutes of a detected CPU spike and gracefully terminated when the load subsided.

6. Challenges and Solutions

- **Scalability Challenges:** Meeting dynamic demand was resolved by deploying the ASG across two Availability Zones (ap-south-1a and ap-south-1b), ensuring fault tolerance.
- **Security & Compliance:** Prevented direct internet access to EC2 instances by placing them in private subnets and only allowing inbound traffic from the ALB.

7. Results and Discussion

- **Performance Gains:** The ALB successfully distributed traffic equally among healthy targets. During load testing, the ASG successfully scaled out from 2 to 5 instances, preventing application downtime.
- **Cost Optimization:** By configuring dynamic scaling policies, resources are only provisioned when necessary, significantly reducing costs during low-traffic periods.

8. Lessons Learned

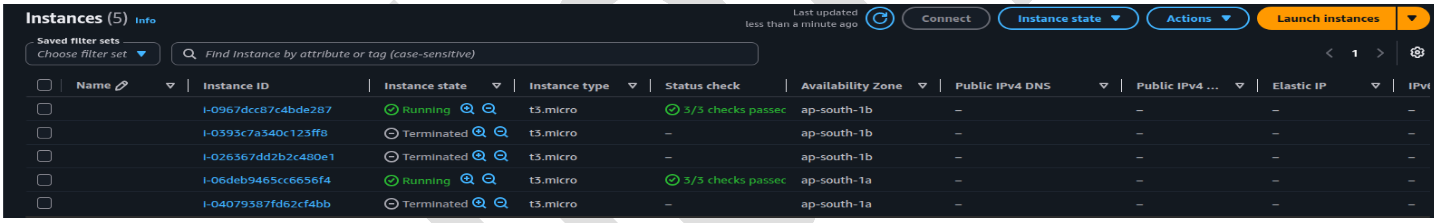
- Insights gained include the critical importance of decoupling compute and storage (using EFS) so that auto-scaled instances always serve synchronized data.
- Verified the necessity of health checks at both the ALB and ASG levels for a self-healing architecture.

9. Conclusion and Recommendations

The deployment meets all objectives for high availability, security, and automated scalability. For future iterations, implementing a CloudFront CDN and AWS WAF (Web Application Firewall) could further enhance performance and security.

10. Appendices

Baseline Performance (Normal State)



Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv
	i-0967dcc87c4bde287	Running	t3.micro	3/3 checks passec	ap-south-1b	–	–	–	–
	i-0393c7a340c123ff8	Terminated	t3.micro	–	ap-south-1b	–	–	–	–
	i-026367dd2b2c480e1	Terminated	t3.micro	–	ap-south-1b	–	–	–	–
	i-06deb9465cc6656f4	Running	t3.micro	3/3 checks passec	ap-south-1a	–	–	–	–
	i-04079387fd62cf4bb	Terminated	t3.micro	–	ap-south-1a	–	–	–	–

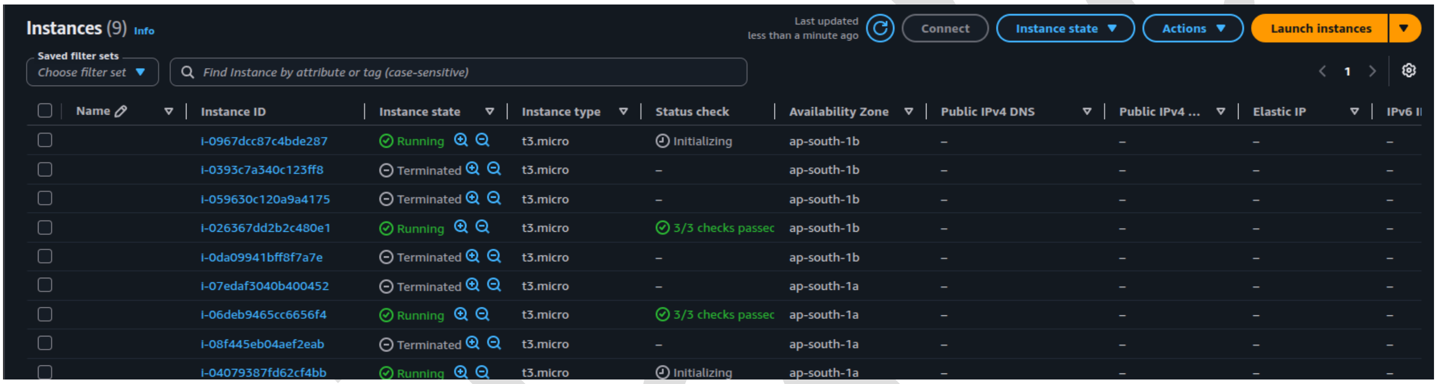
CPU Utilization Spike

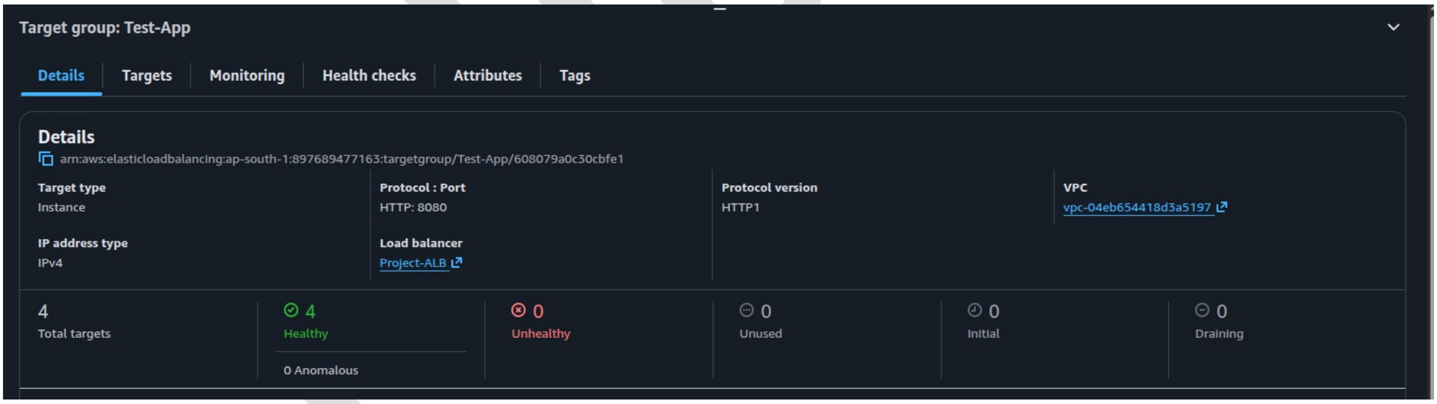
CloudWatch CPU Utilization graph showing the spike during the 'stress' command load testing.

Auto Scaling Activity History

ASG Activity history showing new EC2 instances being launched automatically.

Scaled-Out Instances & Target Group





Target Group showing all 4 instances in a "Healthy" state.